

Manual Tecnico CleanHome

Guia de instalacion, ejecucion, arquitectura e integracion local

Fecha de actualizacion: 23 de mayo de 2026

Proyecto: CleanHome | Backend: Node.js + Express + MySQL | Android: Kotlin + Retrofit + SQLite | Web: HTML/CSS/JS

1. Proposito

Este manual tecnico describe como preparar, levantar y validar el proyecto CleanHome de extremo a extremo. Incluye base de datos MySQL, backend REST, aplicativo web administrativo y aplicativo Android con almacenamiento local SQLite.

El documento tambien resume arquitectura, estructura de carpetas, endpoints, variables de entorno, flujo de pruebas, mantenimiento y consideraciones de seguridad.

2. Alcance del sistema

- * Backend REST para autenticacion, catalogo de servicios, solicitudes y administracion.
- * Base central MySQL como fuente oficial de datos.
- * Frontend web administrativo para gestionar servicios, solicitudes y asignaciones.
- * Aplicacion Android para clientes y panel admin basico.
- * SQLite local en Android para cache de servicios, sesion e historial.
- * Preparacion para despliegue futuro cambiando configuracion de URL y ambiente.

3. Arquitectura general

Android App -> Backend Express -> MySQL

Admin Web -> Backend Express -> MySQL

El backend es la capa central. Android y el panel web no acceden directamente a MySQL; consumen endpoints HTTP bajo /api. Android usa SQLite solo como almacenamiento local de apoyo.

4. Requisitos de ambiente

- * Node.js y npm instalados.
- * MySQL Server instalado y en ejecucion.
- * Base de datos creada desde cleanhome-backend/database/cleanhome.sql.
- * Android Studio instalado para compilar y ejecutar cleanHome-androidApp.
- * Emulador Android o dispositivo fisico en la misma red que la PC.
- * Git instalado para control de versiones.

5. Estructura del repositorio

CleanHome/

- cleanhome-backend/	API REST Node.js + Express
- cleanhome-admin-frontend/	Panel web administrativo

- cleanHome-androidApp/	Aplicacion Android Kotlin/XML
- cdocumentacion/	Fuente Word del manual tecnico
- documentacion/	PDF del manual tecnico
- tools/	Scripts auxiliares
`- README.md	Guia general del proyecto

6. Base de datos MySQL

Si no hay datos relevantes, la opcion recomendada es recrear la base desde cero usando el script principal.

```
mysql -u TU_USUARIO -p
DROP DATABASE IF EXISTS CleanHomeDB;
CREATE DATABASE CleanHomeDB CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
exit
mysql -u TU_USUARIO -p CleanHomeDB < cleanhome-backend/database/cleanhome.sql
```

Si la base ya existia y se desea migrar, aplicar una sola vez:

```
mysql -u TU_USUARIO -p CleanHomeDB < cleanhome-backend/database/migrations/001_sync_support.sql
```

Tablas principales: roles, usuarios, servicios, personal_limpieza y solicitudes. La tabla solicitudes incluye client_temp_id para evitar duplicados en sincronizacion movil.

7. Configuracion del backend

Crear cleanhome-backend/.env. Este archivo no debe subirse a Git.

```
PORT=3000
DB_HOST=localhost
DB_USER=TU_USUARIO
DB_PASSWORD=TU_PASSWORD
DB_NAME=CleanHomeDB
JWT_SECRET=CAMBIAR_EN_CADA_AMBIENTE
JWT_EXPIRES_IN=8h
```

Para levantar el backend:

```
cd cleanhome-backend
npm install
npm start
```

URL esperada: <http://localhost:3000/api>

Validacion rapida: abrir <http://localhost:3000/api/servicios> y confirmar una respuesta JSON con servicios.

8. Endpoints principales

* POST /api/auth/register - Registro de cliente.

- * POST /api/auth/login - Login y retorno de JWT.
- * GET /api/servicios - Catalogo publico de servicios activos.
- * GET /api/servicios/:id - Detalle de servicio.
- * POST /api/solicitudes - Crear solicitud autenticada.
- * GET /api/solicitudes/mis-solicitudes - Historial del cliente autenticado.
- * POST /api/solicitudes/sync - Sincronizacion de solicitudes offline.
- * GET /api/admin/solicitudes - Solicitudes para admin.
- * PATCH /api/admin/solicitudes/:id/estado - Cambio de estado.
- * PATCH /api/admin/solicitudes/:id/asignar-personal - Asignacion de personal.
- * GET /api/admin/servicios - Lista admin de servicios.
- * POST /api/admin/servicios - Crear servicio.
- * PUT /api/admin/servicios/:id - Editar servicio.
- * DELETE /api/admin/servicios/:id - Desactivar servicio.

9. Aplicativo web administrativo

El panel web se encuentra en cleanhome-admin-frontend y se sirve como frontend estatico mediante Node.js.

```
cd cleanhome-admin-frontend
npm install
npm start
```

URL esperada: http://localhost:5173

- * Login con cuenta de rol Admin.
- * Listado de solicitudes.
- * Cambio de estado de solicitud.
- * Asignacion de personal de limpieza.
- * CRUD logico de servicios: crear, editar y desactivar.
- * La URL base de API se configura desde la pantalla de login.

10. Aplicativo Android

La app Android esta en cleanHome-androidApp. Se ejecuta desde Android Studio y consume la API usando Retrofit.

Archivo de configuracion de API:

```
cleanHome-androidApp/app/src/main/java/.../api/ApiConfig.kt
```

Para emulador Android usar:

```
const val BASE_URL = "http://10.0.2.2:3000/api/"
```

Para dispositivo fisico usar una IP local de la PC solo en el entorno local, por ejemplo el formato http://IP_DE_TU_PC:3000/api/. No subir IPs personales al repositorio.

- * Registro y login de cliente.
- * Guardado de token JWT y datos de sesion.

- * Catalogo de servicios desde backend.
- * Agenda con selectores nativos de fecha y hora.
- * Historial con fecha corta y colores por estado.
- * Perfil con navegacion de regreso y cierre de sesion.
- * Panel admin Android para listar solicitudes, cambiar estado y administrar servicios.

11. SQLite local Android

SQLite local no reemplaza MySQL. Su funcion es conservar datos utiles en el dispositivo y preparar soporte offline.

- * usuario_sesion: usuario autenticado y token.
- * servicios: cache del catalogo.
- * solicitudes: historial local.
- * solicitudes_pendientes_sync: cola para solicitudes creadas sin red.
- * sync_metadata: marcas de ultima sincronizacion por modulo.

Archivo de referencia: cleanhome-backend/database/sqlite_android_cleanhome.sql

12. Flujo de levantamiento paso a paso

1. Iniciar MySQL Server.
2. Crear o actualizar CleanHomeDB con los scripts SQL.
3. Configurar cleanhome-backend/.env.
4. Ejecutar npm install y npm start en cleanhome-backend.
5. Validar <http://localhost:3000/api/servicios>.
6. Ejecutar npm install y npm start en cleanhome-admin-frontend.
7. Abrir <http://localhost:5173> y validar login admin.
8. Crear un servicio desde el panel web.
9. Abrir cleanHome-androidApp en Android Studio.
10. Confirmar ApiConfig.kt con 10.0.2.2 para emulador.
11. Ejecutar la app Android.
12. Registrar o iniciar sesion con cliente.
13. Confirmar que el servicio creado en web aparece en catalogo Android.
14. Agendar una solicitud desde Android.
15. Validar la solicitud en historial Android y en panel admin web.

13. Validacion de integracion

- * GET /api/servicios responde desde navegador o Postman.
- * El panel web crea servicios y se reflejan en MySQL.
- * Android carga los servicios creados desde web.
- * Android crea solicitudes con JWT valido.

- * El historial Android muestra la solicitud con estado y fecha legible.
- * El panel admin web o Android muestra la solicitud creada.
- * El cambio de estado se refleja posteriormente en el historial.

14. Seguridad y buenas practicas

- * No subir .env, local.properties, node_modules ni carpetas build.
- * No subir IPs personales; usar placeholders o 10.0.2.2 para emulador.
- * Cambiar JWT_SECRET por ambiente.
- * Cambiar credenciales demo antes de un despliegue real.
- * No guardar contraseñas en texto plano; el backend usa bcrypt.
- * Usar HTTPS en despliegues reales.
- * Configurar CORS segun dominios reales en produccion.

15. Mantenimiento

- * Actualizar README.md cuando cambie la arquitectura o los puertos.
- * Actualizar cleanhome-backend/README.md cuando cambien endpoints o variables de entorno.
- * Actualizar cleanHome-androidApp/README.md cuando cambie la URL base, SQLite o flujos Android.
- * Crear migraciones SQL para cambios en bases existentes.
- * Probar backend, web y Android antes de hacer merge a main.

16. Solucion de problemas

- * Android no conecta al backend: Usar 10.0.2.2 en emulador. En dispositivo fisico usar IP local de la PC y revisar firewall.
- * Login falla: Verificar usuario, rol, hash de contraseña y JWT_SECRET.
- * Servicios no aparecen: Validar que servicios.activo sea verdadero y que GET /api/servicios responda.
- * Solicitud no se crea: Revisar token JWT, campos obligatorios y logs del backend.
- * Panel web no carga datos: Confirmar API Base en login y que backend este en ejecucion.
- * Fecha larga en Android: Usar formateo de UI en adapters; el backend puede devolver timestamps completos.

17. Cierre

Con este flujo, CleanHome queda listo para ejecutarse localmente con MySQL, backend Express, panel web administrativo y app Android integrada. Para un despliegue futuro, el cambio central sera publicar backend/base de datos y actualizar la URL base consumida por web y Android.